

OpenClaw实战课：零基础开发AI自动化运维工具

讲师：韩先超 🙌 云计算架构师、中国计算机学会人工智能与模式识别委员

《Kubernetes从入门到DevOps企业应用实战》作者

韩先超

- ✓ 51CTO学堂K8S、Python教学总监。
- ✓ 51CTO学堂年度十大杰出讲师、人气讲师、金牌讲师合伙人
- ✓ 阿里云开发者社区专家博主、腾讯云架构师北京同盟成员、ITPUB年度影响人物
- ✓ 中国计算机学会（CCF）任职人工智能与模式识别委员、计算机应用委员
- ✓ Linux基金会开源软件学园2023、2024年度优秀讲师
- ✓ 云计算架构师。百度搜索韩先超可以搜到个人信息
- ✓ 清华大学出版社出版的【Kubernetes从入门到DevOps企业应用实战】畅销书作者。
- ✓ 具有丰富的在线教育经验，同时发表过多篇Linux、DevOps、K8s、自动化等方面的系列文章，累计阅读量现已突破100万。
- ✓ 参加过新一代云计算大会，作为主讲嘉宾分享K8s如何助力企业转型、被北京日报发文专题报道。
- ✓ 曾为中铁第四勘察设计院、中国铁道科学研究院、兴业银行、宁波银行、广东移动、河北石家庄移动、安徽合肥电信、深圳任子行、银川电网、咪咕、天翼物联等大型企业提供K8S、Go、Python、AI等内部培训和架构设计。



1、抖音号：hxck8s1

定期免费直播和发布技术视频

2、视频号：扫描下方二维码关注



3、韩先超老师微信公众号，定期发布技术类文章

韩老师微信号：luckylucky421302



韩先超：云计算架构师、51CTO学堂K8s教学总监

扫描右侧二维码添加我微信

邀请大家进学习群

发免费学习资料和视频

随时帮助大家解决生产问题和课程学习遇到的各类问题 微信号：luckylucky421302



加微信备注精品班学员以及班级和名字

kubernetes从入门到DevOps企业应用实战畅销书作者



1/5

¥64.8

已售 31

【当当】Kubernetes 从入门到 DevOps 企业应用实战

韩先超讲技术 /k8s/lin... 推荐

保障 安心购 7天无理由退货·运费险

物流 现货 48小时内从北京市发货，包邮

当当卖场旗舰店

店逛逛

进店

客服

购物车

加入购物车

立即购买

京东搜韩先超，可以搜到

课程核心内容：

- ✓ OpenClaw (龙虾) 安装和使用
- ✓ 从零开始搭建AI开发环境
- ✓ AI自动化编程Agent大全
- ✓ 零基础开发AI自动化运维工具-管理Linux服务
- ✓ 开发自动化工具-智能管理tomcat服务
- ✓ 开发自动化工具-智能管理Httpd服务
- ✓ 开发自动化工具-智能管理k8s集群
- ✓ 开发自动化工具-智能管理Mysql服务
- ✓ 基于 LLM 构建K8S故障智能诊断系统
- ✓ 基于 EFK + LLM 的智能日志诊断系统

✓ OpenClaw (龙虾) 安装和使用

OpenClaw (龙虾) 介绍

OpenClaw (曾用名 **Clawdbot**、**Moltbot**) 是一款开源 AI Agent 智能体框架, 通过整合 **大语言模型 (LLM) + 自动化执行能力 + 多渠道通信接口**, 构建一个具备:

- 持久记忆
 - 自动执行任务
 - 多平台通信
 - 本地私有化部署
- 能力的 **智能AI助手系统**。

简单理解:

OpenClaw = 大模型 + Agent + 自动化执行工具

它可以让 AI 不只是“聊天”, 而是能够:

- 自动执行任务
- 调用脚本
- 管理系统
- 与用户持续交互

所以很多人把它当成: **个人AI助手 / AI自动化系统 / AI运维助手**

OpenClaw (龙虾) 核心架构

1 大语言模型 (LLM)

负责:

- 理解用户问题
- 生成任务计划
- 决策执行步骤

常见支持:

- DeepSeek
- OpenAI
- Ollama本地模型
- Claude

OpenClaw (龙虾) 核心架构

2 Agent任务执行模块

Agent负责：

- 解析任务
- 调用工具
- 执行自动化流程

例如：

用户输入：帮我检查服务器CPU使用率

Agent执行流程：

- 1 用户请求
- 2 LLM分析任务
- 3 调用工具脚本
- 4 获取数据
- 5 返回结果

OpenClaw (龙虾) 核心架构

3 工具系统 (Tools)

OpenClaw支持调用各种工具:

例如:

- Shell命令
- Python脚本
- API接口
- 自动化脚本

例如:

- 检查服务器状态
- 重启服务
- 查询日志
- 执行自动化运维

OpenClaw（龙虾）核心架构

4 记忆系统（Memory）

OpenClaw支持 **持久化记忆**：

AI可以记住：

- 用户习惯
- 历史任务
- 执行结果
- 系统状态

这让AI变成 **长期助手**。

OpenClaw（龙虾）核心架构

5 多渠道通信

OpenClaw可以接入很多平台：

例如：

- Web UI
- Telegram
- Discord
- API接口

实现 **随时调用AI助手**。

OpenClaw能做什么

OpenClaw = 大模型 + 自动化工具 + Agent执行逻辑

它可以让 AI 在你的服务器上执行各种任务，比如：

- 自动执行脚本
- 自动巡检linux机器
- 自动检查k8s故障
- 自动调整Prometheus监控告警阈值
- 自动搜索资料
- 自动管理服务器
- 自动调用 API
- 自动处理文件
- 等等

甚至可以接入各种聊天平台，例如：

- 飞书
- Telegram
- Discord
- Slack
- 企业微信
- 钉钉群

让 AI 变成你的自动化助手。更关键的一点是：

OpenClaw 完全可以本地部署，而且安装非常简单。

OpenClaw安装

Linux 服务器准备

建议准备一台 Linux 服务器即可，例如：

- Ubuntu
- Debian
- Rocky Linux
- CentOS

配置建议：

- 内存：4GB 以上更稳定
- Node.js：22 以上版本

如果服务器只有 **2GB 内存**，建议配置 **swap 交换空间**，否则可能会出现 OOM 内存溢出。

OpenClaw安装

安装 Git:

```
yum install git -y
```

安装 Node.js (推荐使用 NVM)

OpenClaw 需要 Node.js 22 以上版本。

推荐使用 **NVM 管理 Node 版本**。

如果是国内服务器，可以使用镜像源：

```
curl -o- https://gitee.com/RubyMetric/nvm-cn/raw/main/install.sh | bash
```

安装完成后重新加载环境变量：

```
source /root/.bashrc
```

安装 Node.js 22:

```
nvm install 22
```

OpenClaw安装

一键安装 OpenClaw

OpenClaw 官方提供了一键安装脚本。

直接执行：

```
curl -fsSL https://openclaw.bot/install.sh | bash
```

实际安装时你会看到类似输出：

OpenClaw Installer

Detected: linux

Preparing environment

✓ Node.js v22.22.1 found

Installing OpenClaw

✓ Git already installed

✓ OpenClaw npm package installed

接下来参考视频安装即可。

OpenClaw安装

Web 控制台

安装完成后，系统会生成一个 **Web 控制台地址**：

`http://127.0.0.1:18789`

不过很多服务器没有图形界面，所以需要通过 **SSH 端口转发** 访问。

Windows 访问 OpenClaw 控制台

在 Windows 上：

按下：Win + R

输入：cmd

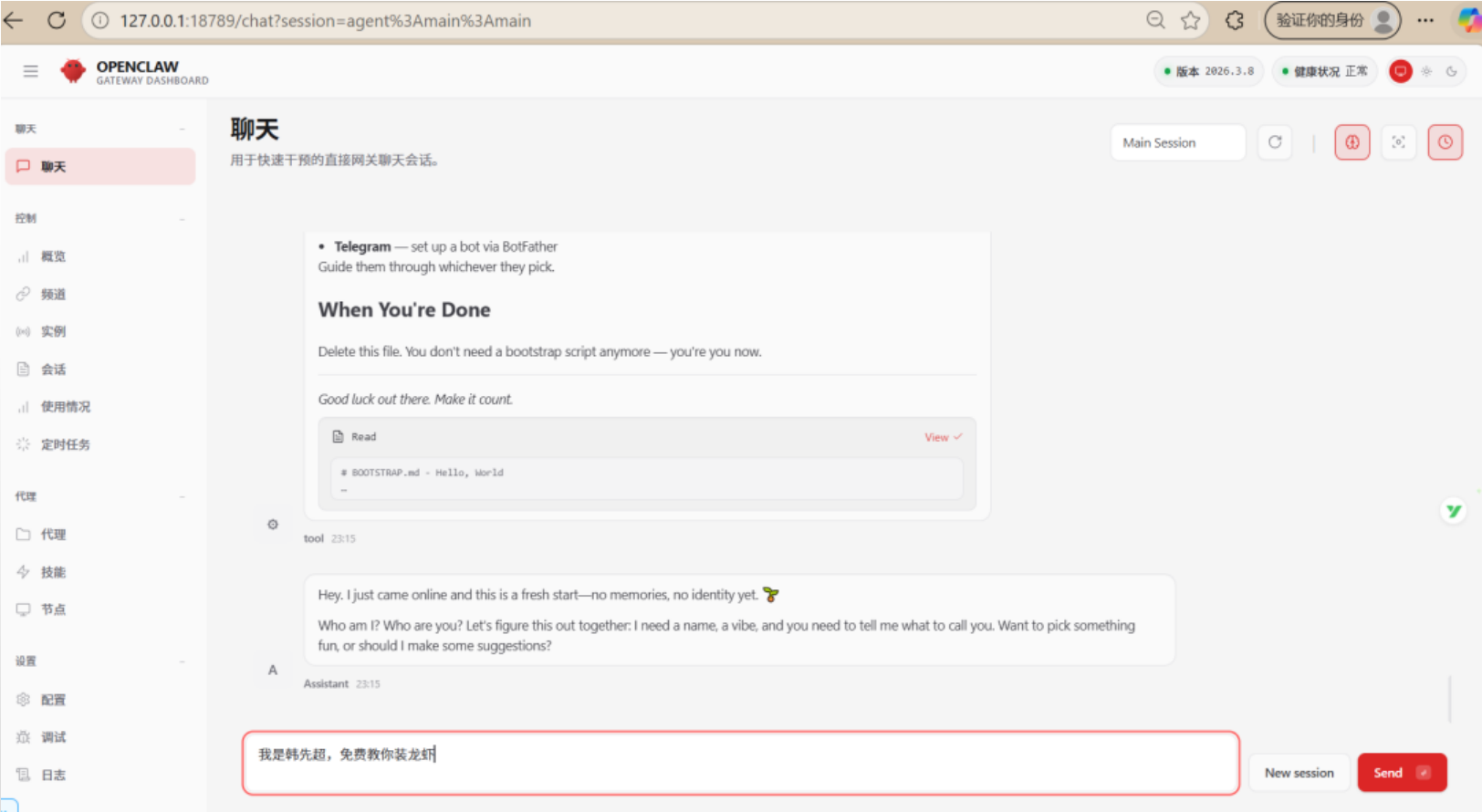
然后执行：`ssh -N -L 18789:127.0.0.1:18789 root@服务器IP`

例如：`ssh -N -L 18789:127.0.0.1:18789 root@192.168.40.190`

然后在浏览器打开：`http://127.0.0.1:18789/#token=13f964c031ffcf0e5f48599f7a7833be48a03c94b6e0960`

OpenClaw安装

就可以看到 OpenClaw 的 Web 控制台界面。



AI时代，运维如何转型

运维发展的八大阶段

✓ 第一阶段：传统基础运维

- **关键词：**体力活、手动部署
- **工作内容：**机房上架服务器、系统安装、调网络、部署Nginx/MySQL等服务
- **技术点：**Linux命令操作、Shell 脚本、TCP/IP、中间件运维

✓ 第二阶段：虚拟化运维

- **关键词：**资源池化
- **工作内容：**使用 KVM/OpenStack 实现服务器虚拟化
- **技术点：**OpenStack、Libvirt、KVM、虚拟交换机（OVS）

✓ 第三阶段：云运维

- **关键词：**弹性计算、云API
- **工作内容：**ECS 创建、云数据库、云安全组等资源调度
- **技术点：**阿里云、腾讯云、Terraform、Boto3（AWS SDK）

运维发展的八大阶段

✓ 第四阶段：容器化运维

- **关键词：**环境一致性、轻量打包
- **工作内容：**使用 Docker 打包服务和应用，提升部署效率
- **技术点：**Docker、Dockerfile、镜像仓库（Harbor）

✓ 第五阶段：Kubernetes 运维

- **关键词：**自动调度、自恢复、弹性伸缩
- **典型任务：**部署 K8s，运行微服务，进行灰度发布、负载均衡
- **关键技术：**Kubernetes、Helm、Ingress、Prometheus、Istio等

✓ 第六阶段：自动化运维（DevOps）

- **关键词：**持续集成、持续交付、一键上线
- **典型任务：**打通代码 → 构建 → 发布全链路
- **关键技术：**
 - 1) 流水线工具：Jenkins、Tekton、ArgoCD
 - 2) 配置工具：Ansible、SaltStack、Terraform

运维发展的八大阶段

✓ 第七阶段：Python 运维开发

- **关键词：**平台开发、脚本智能、运维工具化
- **典型任务：**
 - 1) 编写自动化脚本和管理工具
 - 2) 开发 Web 面板自动管理服务（如 nginx、tomcat、K8s 管理界面）
 - 3) 接入 API、批量处理、资源监控
- **关键技术：**
 - 1) Python 标准库（os、subprocess、psutil）
 - 2) Web 框架（Flask、FastAPI、Django）
 - 3) 运维工具库（paramiko、requests、tkinter等）

✓ 第八阶段：智能运维（AIOps）

- **关键词：**无人值守、AI辅助决策、自愈系统
- **典型任务：**
 - 1) 监控指标/日志智能分析，异常检测和自动修复
 - 2) 自动重启高负载进程、生成邮件告警
- **关键技术：**
 - 1) Python
 - 2) AI 大模型：Scikit-learn、LangChain、MLFlow、LlamaFactory、Llama.cpp等
 - 2) 开源服务：Prometheus、zabbix、EFK等

AI自动化编程agent大全

开发者使用最多的 AI 编程助手

1、Codegeex

CodeGeeX 是一个 **基于大模型的 AI 编程助手**，可以在 IDE 中自动生成代码、补全代码和解释代码。

它由 **清华大学 THUDM 实验室** 开发，是一个开源 AI 编程工具。

核心模型规模：

- **130亿参数**

训练数据：**8500亿代码 token**

支持 **20+ 编程语言**

CodeGeeX 插件可以安装在：

- VS Code
- IntelliJ IDEA
- PyCharm
- WebStorm

也就是说：**Python + VSCode 可以直接用。**

开发者使用最多的 AI 编程助手

CodeGeeX 核心能力：

1、自动生成代码

写注释 → AI生成代码

示例：

写一个Python脚本，获取Linux CPU和内存

2、代码自动补全

输入：

```
for i in range(
```

AI 自动补全：

```
for i in range(10):
```

```
    print(i)
```

3 代码解释和注释

开发者使用最多的 AI 编程助手

CodeGeeX 优势：

优势1：免费

优势2：国产模型

很多国内开发者更容易使用
支持：

- 中文 prompt
- 中文代码解释

优势3：多语言支持

支持：

- Python
- Java
- C++
- Go
- JavaScript
- 20+ 编程语言。

开发者使用最多的 AI 编程助手

2、GitHub Copilot

最知名的 AI 编程插件（微软+OpenAI）

核心能力：

- 自动补全代码
- 根据注释生成代码
- 自动生成函数
- AI Chat解释代码
- 自动写单元测试

演示案例：# 写一个Python脚本监控Linux CPU

Copilot自动生成：

```
import psutil  
print(psutil.cpu_percent())
```

特点：

- VSCode 原生支持
- Python / Java / Go / JS 都支持
- 自动补全非常强
- 很多公司开发者 **默认必装插件。**

开发者使用最多的 AI 编程助手

3、Windsurf Plugin (formerly Codeium)

Copilot 替代方案

核心能力：

- AI代码自动补全
- AI Chat
- 自动解释代码
- 自动生成函数

优势：

- **完全免费**
- 支持 **40+ IDE**
- 速度快
- 很多个人开发者 **用它替代Copilot。**

开发者使用最多的 AI 编程助手

4、Tabnine

企业级 AI 编程助手

特点：

- 自动补全代码
- 支持本地模型
- 企业级隐私保护

适合：

- 企业开发
- 内网开发环境

AI Agent级编程工具（更强）

这类是 **真正的 AI 编程 Agent**，不仅写代码，还能改代码。

1、Cline

最强 **VSCode AI Agent 插件**

能力：

- 自动修改代码
- 自动创建文件
- 自动执行命令
- 自动重构项目

甚至可以：

帮我写一个Python自动化运维工具

AI会：

- 1 创建目录
- 2 写代码
- 3 调试代码
- 4 修改代码

Cline可以理解整个代码仓库并执行多步骤任务。

AI Agent级编程工具（更强）

2、Continue

开源 AI 编程插件

特点：

支持 GPT

- 支持 Claude
- 支持 DeepSeek
- 支持本地 LLM

3、CodeGPT

特点：

- 直接调用 OpenAI / Claude / DeepSeek
- 支持 RAG
- 支持自定义 Prompt

AI 编程 IDE（不是插件）

这些是 **AI IDE**。

1、Cursor

目前最火 AI IDE

特点：

- AI 深度集成
- 可以理解整个代码库
- AI 自动重构项目

但需要 **换 IDE**。

开发 K8s 故障自动诊断工具（实战）

本模块基于真实生产排障流程，带领学员开发一个 AI 驱动的 Kubernetes 智能诊断系统。

系统自动采集 describe、logs、events、资源指标等现场数据，结合大模型分析能力，输出企业级 SRE 故障报告。

通过该实战案例，深入理解 AIOps 在生产环境中的落地方式与架构设计思路。

采用 **DeepSeek-V3.2 产业级大模型**-><https://platform.deepseek.com/>

- 架构：MoE（混合专家模型）
- 总参数规模：671B（6710亿）
- 推理激活参数：≈37B（320亿）
- 支持 128K 长上下文

意味着：能一次性处理完整 Kubernetes 故障现场数据，实现结构化、可执行的企业级 SRE 诊断输出。

这不是普通聊天模型。

这是：
百亿级参数架构驱动的智能运维能力。

推理能力全球领先

- DeepSeek-V3.2 的目标是平衡推理能力与输出长度，适合日常使用，例如问答场景和通用 Agent 任务场景。在公开的推理类 Benchmark 测试中，DeepSeek-V3.2 达到了 GPT-5 的水平，仅略低于 Gemini-3.0-Pro；相比 Kimi-K2-Thinking，V3.2 的输出长度大幅降低，显著减少了计算开销与用户等待时间。
- DeepSeek-V3.2-Speciale 的目标是将开源模型的推理能力推向极致，探索模型能力的边界。V3.2-Speciale 是 DeepSeek-V3.2 的长思考增强版，同时结合了 DeepSeek-Math-V2 的定理证明能力。该模型具备出色的指令跟随、严谨的数学证明与逻辑验证能力，在主流推理基准测试上的性能表现媲美 Gemini-3.0-Pro（见下表）。更令人瞩目的是，V3.2-Speciale 模型成功斩获 IMO 2025（国际数学奥林匹克）、CMO 2025（中国数学奥林匹克）、ICPC World Finals 2025（国际大学生程序设计竞赛全球总决赛）及 IOI 2025（国际信息学奥林匹克）金牌。其中，ICPC 与 IOI 成绩分别达到了人类选手第二名与第十名的水平。

开发 K8s 故障自动诊断工具（实战）

模型细节

注意: `deepseek-chat` 和 `deepseek-reasoner` 对应模型版本不变, 为 DeepSeek-V3.2 (128K 上下文长度), 与 APP/WEB 版不同。

模型		deepseek-chat	deepseek-reasoner
BASE URL		https://api.deepseek.com	
模型版本		DeepSeek-V3.2 (非思考模式)	DeepSeek-V3.2 (思考模式)
上下文长度		128K	
输出长度		默认 4K, 最大 8K	默认 32K, 最大 64K
功能	Json Output	支持	支持
	Tool Calls	支持	支持
	对话前缀续写 (Beta)	支持	支持
	FIM 补全 (Beta)	支持	不支持
价格	百万tokens输入 (缓存命中)	0.2元	
	百万tokens输入 (缓存未命中)	2元	
	百万tokens输出	3元	

开发 K8s 故障自动诊断工具（实战）

K8s LLM 智能诊断平台（远程企业版）

控制节点 IP

192.168.40.180

SSH 用户名

root

登录方式

密码登录

SSH 密码（密码模式使用）

私钥路径（私钥模式使用）

已连接

连接远程K8s控制节点

DeepSeek API Key

选择 Namespace

default

加载 Namespace

选择 Pod

nfs-provisioner-f5554f99d-bvb4x

开始智能诊断

SSH 连接成功
开始采集远程数据...
调用 DeepSeek 分析中...

===== 智能诊断结果 =====

Kubernetes Pod 故障分析报告

一、故障概述

当前异常现象

NFS Provisioner Pod 出现间歇性重启，最近一次重启发生在 2026-02-24 20:37:31（终止）至 20:37:33（重新启动），间隔仅2秒。Pod 重启计数为3次，表明存在稳定性问题。

关键错误信息

开发 K8s 故障自动诊断工具（实战）

<https://platform.deepseek.com/usage>

打开 DeepSeek 的开发者控制台，注册账号，创建key，需要充值，如果做实验，充值10元即可。

代码：k8s_llmops_gui_secure.py

运行代码前置条件，安装依赖模块

```
pip3 install openai paramiko pyinstaller
```

可以直接运行代码，也可以打包成windows程序，用如下命令

```
pyinstaller --onefile --windowed k8s_llmops_gui_secure.py
```

为什么我要做这个工具？

大家有没有发现一个问题：

在 Kubernetes 生产环境里，排查一个 Pod 故障，标准动作是什么？

我们一定会做这几件事：

kubectl describe pod

kubectl logs

kubectl get events

kubectl top pod

然后：

- 自己分析日志
- 自己判断错误
- 自己查资料
- 自己推断根因

问题来了：

- 👉 如果是新手工程师呢？
- 👉 如果是夜里 2 点值班呢？
- 👉 如果日志非常复杂呢？

这就是我们今天要解决的核心问题：如何用 AI 提升 Kubernetes 故障排查效率？

我们要做一个什么工具？

我们要做一个：K8s + AI 的智能故障排查助手

核心目标：

- 自动采集现场数据
- 自动分析故障
- 输出企业级 SRE 诊断报告

它不是一个聊天机器人。它是一个：面向生产环境的智能 SRE 助手

整体架构思路

设计思路：

GUI 界面



SSH 连接远程控制节点



自动执行 kubectl 命令



采集 describe / logs / events / top



拼接 Prompt



调用 DeepSeek 大模型



输出结构化诊断报告

代码核心模块讲解思路

❶ 第一部分：远程采集现场数据

我们用的是：paramiko 远程 SSH

自动执行 kubectl

核心函数：

```
run_remote_cmd(cmd)
```

这个函数的作用就是：在远程 K8s 控制节点上执行命令

这一步相当于：

👉 模拟人工排查过程

代码核心模块讲解思路

2 第二部分：采集故障现场四件套

在 `diagnose_thread` 里：

```
describe = kubectl describe (查看pod详细信息)
logs_data = kubectl logs (查看pod日志)
events = kubectl get events (查看pod event事件)
top = kubectl top (查看pod 资源使用情况)
```

这四个数据源，就是K8s 排障必备四条命令

代码核心模块讲解思路

3 第三部分：Prompt 工程设计

我们是：用企业级 SRE 报告结构去约束输出

比如：

- 故障概述
- 影响范围
- 根因排序
- 紧急处置措施
- 验证方法
- 长期优化建议

这就叫：结构化 Prompt 设计

这比“问一句：帮我看看怎么回事”高级太多。

代码核心模块讲解思路

4 第四部分：调用大模型

这一段代码：`client.chat.completions.create()`

本质上做的事情是：把 Kubernetes 的实时现场数据交给 AI 分析

这就是：AI + 云原生的结合。

传统运维是“人找问题”，AIOps 是“系统帮你找问题”未来的运维工程师，不是手工执行 `kubectl` 的人，而是设计智能系统的人。

开发企业级日志智能分析平台

本系统基于 EFK 架构，从 Elasticsearch 中提取生产环境日志数据，将 describe、logs、events 等现场信息统一汇聚到es后，传递给 DeepSeek 大模型进行架构级分析。模型输出结构化诊断结果，包括：

- 是否异常
- 风险等级
- 影响范围
- 根因分析
- 应急处理建议
- 架构优化建议

实现从“日志检索”到“智能诊断”的升级。

采用 DeepSeek-V3.2 产业级大模型

- 架构：MoE（混合专家模型）
- 总参数规模：671B（6710亿）
- 推理激活参数：≈37B（320亿）
- 支持 128K 长上下文

推理能力全球领先

- DeepSeek-V3.2 的目标是平衡推理能力与输出长度，适合日常使用，例如问答场景和通用 Agent 任务场景。在公开的推理类 Benchmark 测试中，DeepSeek-V3.2 达到了 GPT-5 的水平，仅略低于 Gemini-3.0-Pro；相比 Kimi-K2-Thinking，V3.2 的输出长度大幅降低，显著减少了计算开销与用户等待时间。
- DeepSeek-V3.2-Speciale 的目标是将开源模型的推理能力推向极致，探索模型能力的边界。V3.2-Speciale 是 DeepSeek-V3.2 的长思考增强版，同时结合了 DeepSeek-Math-V2 的定理证明能力。该模型具备出色的指令跟随、严谨的数学证明与逻辑验证能力，在主流推理基准测试上的性能表现媲美 Gemini-3.0-Pro（见下表）。更令人瞩目的是，V3.2-Speciale 模型成功斩获 IMO 2025（国际数学奥林匹克）、CMO 2025（中国数学奥林匹克）、ICPC World Finals 2025（国际大学生程序设计竞赛全球总决赛）及 IOI 2025（国际信息学奥林匹克）金牌。其中，ICPC 与 IOI 成绩分别达到了人类选手第二名与第十名的水平。

开发企业级日志智能分析平台

模型细节

注意: `deepseek-chat` 和 `deepseek-reasoner` 对应模型版本不变, 为 DeepSeek-V3.2 (128K 上下文长度), 与 APP/WEB 版不同。

模型		deepseek-chat	deepseek-reasoner
BASE URL		https://api.deepseek.com	
模型版本		DeepSeek-V3.2 (非思考模式)	DeepSeek-V3.2 (思考模式)
上下文长度		128K	
输出长度		默认 4K, 最大 8K	默认 32K, 最大 64K
功能	Json Output	支持	支持
	Tool Calls	支持	支持
	对话前缀续写 (Beta)	支持	支持
	FIM 补全 (Beta)	支持	不支持
价格	百万tokens输入 (缓存命中)	0.2元	
	百万tokens输入 (缓存未命中)	2元	
	百万tokens输出	3元	

开发企业级日志智能分析平台

企业级 AI 运维分析平台 (最终稳定版)

ES IP: 192.168.40.180

端口: 30920

加载索引: logstash-*

时间范围: 2天

DeepSeek Key: *****

测试ES连接

开始分析

导出Word

导出PDF

[2026-02-24T14:00:44.970076062+00:00] time="2026-02-24T14:00:44Z" level=error msg="ERROR: diskstats collector failed after 0.000170s: invalid line for /host/proc/diskstats for sda" source="collector.go:132"

[2026-02-24T14:00:45.330761115+00:00] E0224 14:00:45.330099 1 horizontal.go:275 "Unhandled Error" err="failed to query scale subresource for Deployment/default/php-apache: deployments/scale.apps \"php-apache\" not found" logger="UnhandledError"

[2026-02-24T14:00:45.330839962+00:00] E0224 14:00:45.330264 1 horizontal.go:275 "Unhandled Error" err="failed to query scale subresource for Deployment/default/nginx-shop: deployments/scale.apps \"nginx-shop\" not found" logger="UnhandledError"

[2026-02-24T14:00:45.831897558+00:00] 2026-02-24 14:00:45.831 [INFO] [83] monitor-addresses/autodetection_methods.go 117: Using autodetected IPv4 address 192.168.40.180/24 on matching interface ens33

[2026-02-24T14:00:45.611272746+00:00] 2026-02-24 14:00:45 +0000 [info]: #0 [filter_kube_metadata] stats - namespace_cache_size: 3, pod_cache_size: 7, namespace_cache_api_updates: 7, pod_cache_api_updates: 7, id_cache_miss: 7

[2026-02-24T14:00:45.942650835+00:00] E0224 14:00:45.942034 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1beta1.CronJob: the server could not find the requested resource

[2026-02-24T14:00:45.981131766+00:00] E0224 14:00:45.980615 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1beta1.VolumeAttachment: volumeattachments.storage.k8s.io is forbidden: User "system:serviceaccount:kube-system:kube-state-metrics" cannot list resource "volumeattachments" in API group "storage.k8s.io" at the cluster scope

[2026-02-24T14:00:45.987387722+00:00] E0224 14:00:45.986664 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v2beta1.HorizontalPodAutoscaler: the server could not find the requested resource

[2026-02-24T14:00:45.988852507+00:00] E0224 14:00:45.988258 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1beta1.PodDisruptionBudget: poddisruptionbudgets.policy is forbidden: User "system:serviceaccount:kube-system:kube-state-metrics" cannot list resource "poddisruptionbudgets" in API group "policy" at the cluster scope

[2026-02-24T14:00:46.008242440+00:00] E0224 14:00:46.004351 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1.Deployment: deployments.apps is forbidden: User "system:serviceaccount:kube-system:kube-state-metrics" cannot list resource "deployments" in API group "apps" at the cluster scope

[2026-02-24T14:00:46.066818753+00:00] E0224 14:00:46.066172 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1.Secret: secrets is forbidden: User "system:serviceaccount:kube-system:kube-state-metrics" cannot list resource "secrets" in API group "" at the cluster scope

[2026-02-24T14:00:46.071057042+00:00] E0224 14:00:46.070468 1 reflector.go:156] pkg/mod/k8s.io/client-go@v0.0.0-20191109102209-3c0d1af94be5/tools/cache/reflector.go:108: Failed to list *v1.Namespace: namespaces.networking.k8s.io is forbidden: User "system:serviceaccount:kube-system:kube-state-metrics" cannot list resource "namespaces" in API group "networking.k8s.io" at the cluster scope

===== 企业级 AI 运维分析报告 =====

【是否异常】

是

风险等级: 中

【影响范围】

1. 监控与指标收集系统 (kube-state-metrics, node-exporter, Prometheus)。

2. 水平自动扩缩容 (HPA) 功能。

3. 集群资源 (如Deployment、Secret、ConfigMap等) 的监控视图。

【故障类型】

1. 权限配置错误 (RBAC)。

2. API版本不兼容或资源不存在。

3. 监控数据收集异常。

【根因分析】

1. kube-state-metrics服务账号 (system:serviceaccount:kube-system:kube-state-metrics) 缺少必要的ClusterRole权限, 导致无法列出Deployment、Secret、ConfigMap等多种集群资源。

2. 集群中可能未启用或不存在某些API资源 (如v2beta1.HorizontalPodAutoscaler、v1beta1.CronJob), 导致API Server返回"the server could not find the requested resource"。

3. node-exporter在读取 /host/proc/diskstats 时遇到无效行, 可能是主机磁盘状态文件格式异常或访问路径问题。

4. HPA控制器无法查询特定Deployment (php-apache, nginx-shop) 的scale子资源, 表明这些Deployment可能已被删除或不存在。

【应急处理建议】

1. 检查并修复kube-state-metrics的RBAC配置, 确保其服务账号拥有必要的list权限。

2. 验证集群中是否存在日志中报错的Deployment (如default/php-apache, default/nginx-shop), 若不存在需重新创建或调整HPA配置。

3. 检查API Server的API版本, 确保使用的资源版本 (如v2beta1.HorizontalPodAutoscaler) 与集群版本兼容。

4. 检查node-exporter的hostPath挂载, 确保 /host/proc/diskstats 路径有效且可读。

【架构优化建议】

1. 统一并标准化集群的RBAC权限管理, 定期审计服务账号权限。

2. 升级监控组件 (如kube-state-metrics、Prometheus) 以适应集群Kubernetes版本, 避免API版本不兼容。

3. 对HPA目标资源进行健康检查, 避免指向不存在的Deployment。

4. 考虑使用更稳定的监控数据收集方案, 如通过DaemonSet部署node-exporter并确保主机路径挂载正确。

开发企业级日志智能分析平台

<https://platform.deepseek.com/usage>

打开 DeepSeek 的开发者控制台，注册账号，创建key，需要充值，如果做实验，充值10元即可。

代码：log_ai_analyzer.py

运行代码前置条件，安装依赖模块

```
pip3 install elasticsearch==7.17.9 python-docx reportlab matplotlib requests openai paramiko pyinstaller
```

可以直接运行代码，也可以打包成windows程序，用如下命令

```
pyinstaller --onefile --windowed log_ai_analyzer.py
```

整体架构

这段程序可以分成 6 大模块：

GUI界面层



ES连接层



日志查询层



AI分析层



结果展示层



报告导出层 (Word / PDF)

是一个典型的：**日志采集 + LLM分析 + 报告自动生成** 的桌面运维工具

核心模块详解

1 配置区

```
DEFAULT_ES_IP = "192.168.40.180"  
DEFAULT_ES_PORT = "30920"  
DEFAULT_ES_INDEX = "logstash-*"  
DEEPSEEK_BASE_URL = "https://api.deepseek.com"  
DEEPSEEK_MODEL = "deepseek-chat"
```

作用：

- 设置默认 Elasticsearch 地址
- 默认索引模式
- DeepSeek API 基础地址
- 使用 deepseek-chat 模型

这个设计是对的：

- IP 可修改
- 索引可选
- 时间范围可选

核心模块详解

2 LogAIApp 类（主程序）

这是整个系统的核心类。

初始化函数：

```
def __init__(self, root):
```

作用：

- 设置窗口标题
- 设置窗口大小
- 初始化 ES 对象
- 初始化 AI 客户端
- 构建UI

核心模块详解

3 UI 构建模块

def build_ui(self):

构建界面包含：

控件	功能
ES IP 输入框	可修改地址
端口输入框	可修改端口
加载索引按钮	获取 ES 所有索引
时间选择	1天/2天/7天/30天
API Key 输入	DeepSeek Key
测试ES	ping
开始分析	主流程
导出Word	生成docx
导出PDF	生成pdf
日志显示区	原始日志
结果区	AI报告

核心模块详解

4 ES连接模块

```
test_es():
```

```
    es.ping() #判断ES是否连通
```

```
load_indices():
```

```
    es.indices.get_alias("*") #拉取所有索引
```

```
self.index_combo["values"] = indices #更新下拉框
```

5 启动分析流程

```
start():
```

```
    threading.Thread(target=self.run)
```

```
#为什么要用线程？ 如果不用线程， GUI会卡死
```

核心模块详解

6 run() 主流程:

执行顺序:

清空界面



初始化ES



初始化AI



查询日志



显示日志



调用大模型



解析JSON



展示报告

核心模块详解

7 日志查询模块

```
def fetch_logs(self):
```

第一步：获取时间范围

```
days = int(self.days_combo.get().replace("天", ""))
```

用户选 7天 → 转成整数7

第二步：计算时间区间

```
end = datetime.utcnow()
start = end - timedelta(days=days)
```

第三步：自动识别时间字段

```
for field in ["@timestamp", "timestamp", "logtime", "time"]:
```

第四步：构造 range 查询

```
"range": {
  "time_field": {
    "gte": gte,
    "lte": lte
  }
}
```

如果失败：自动降级成普通查询

核心模块详解

8 AI 分析模块

```
def analyze_with_ai(self, logs):
```

核心逻辑：

- 拼接日志
- 构造Prompt
- 调用DeepSeek

```
resp = self.client.chat.completions.create(...)
```

温度设为： temperature=1.3

Temperature 设置

temperature 参数默认为 1.0。

- 我们建议您根据如下表格，按使用场景设置 temperature。

场景	温度
代码生成/数学解题	0.0
数据抽取/分析	1.0
通用对话	1.3
翻译	1.3
创意类写作/诗歌创作	1.5

核心模块详解

9 JSON 解析

```
def parse_json(self, text):
```

先用正则去掉：```json

然后：json.loads(text)

防止模型输出Markdown

10 结果美化输出

```
display_pretty():
```

特点：

- 高风险 → 红色
- 中风险 → 橙色
- 分区块展示

这是一个简单的报告格式引擎。

核心模块详解

1 1 Word导出

- 创建文档
- 添加标题
- 遍历字段
- 保存文件

高风险时: `heading.runs[0].font.color.rgb = RGBColor(255, 0, 0)`

实现红色标题。

1 2 PDF导出

使用: ReportLab, 并注册中文字体: STSong-Light

使用: UnicodeCIDFont

否则中文会乱码。

完整执行流程总结

用户输入ES信息



测试连接



选择索引



选择时间



点击开始



查询日志



调用DeepSeek



生成结构化JSON



界面展示



导出报告